

Clase teórica 6 : Nivel de Transporte

Tomás F. Melli

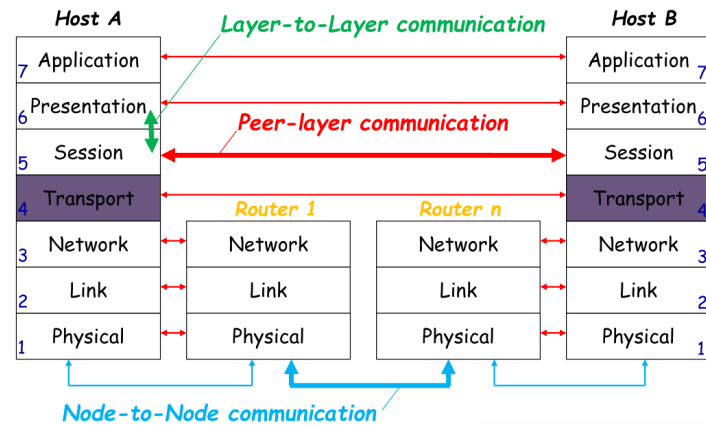
October 2025

Índice

1	Introducción	2
1.1	Enlace de Datos versus Transporte	2
1.2	Máquina de Estados de un Servicio de Transporte	2
1.2.1	Estados del Cliente	3
1.2.2	Estados del Servidor	3
1.3	Protocolos End-to-End en subredes de datagramas	3
2	TCP (Transmission Control Protocol)	4
2.1	Flujo de datos en TCP	4
2.2	Segmentación y MSS	5
2.3	Conexión TCP	5
2.4	Formato del Header TCP	5
2.5	Contexto de encapsulamiento	6
2.6	Emulación del Stream de Bytes	7
2.7	Establecimiento y Liberación de Conexión	8
2.7.1	Establecimiento de Conexión	8
2.7.2	Liberación de Conexión	8
2.8	Ventana Deslizante	8
2.8.1	Ventana Deslizante del Emisor	9
2.8.2	Ventana Deslizante del Receptor	9
2.9	Control de Flujo	10
2.9.1	Funcionamiento	10
2.10	Advertised Window Management y Números de Secuencia	11
2.11	Estados del Modelo TCP	13
2.11.1	Administración de Conexiones	13
2.12	Primitivas – Sockets Berkeley TCP	14
2.12.1	Principales Estados TCP	14
2.13	Ciclo de Vida en la Máquina de Estados TCP	15
2.13.1	TCP Client Lifecycle	15
2.13.2	TCP Server Lifecycle	15
2.14	Liberación de la Conexión TCP (“Tear-down”)	16
3	UDP (User Datagram Protocol)	17
3.1	Multiplexación mediante Puertos	17
3.2	Ejemplo: Transporte UDP y Aplicación RTP	18
4	Retransmisión Adaptativa	18
4.1	TCP Timer Management	19
4.2	Algoritmo Original de Retransmisión Adaptativa	19
4.3	Problemas del Algoritmo Básico y Solución de Karn/Partridge (KP87)	20
4.4	Algoritmo de Jacobson/Karels (JK88)	20
5	Uso del Ancho de Banda, Sockets y Ventana Deslizante	20
5.1	Ventana Deslizante (Sliding Window)	21
5.2	Ejemplo: Delay $\times$ Bandwidth y Ventana TCP	21
5.3	Interpretación de la tabla	21
5.4	Uso de Sockets	22

1 Introducción

Vamos a recordar las capas del modelo OSI mirando este esquema:



En la capa de **Transporte**, se manejan unidades llamadas **TPDU (Transport Protocol Data Unit)**, que representan la información intercambiada entre entidades de transporte de extremo a extremo.

1.1 Enlace de Datos versus Transporte

Mientras que la **capa de Enlace de Datos** se ocupa principalmente de la transmisión confiable entre nodos adyacentes, la **capa de Transporte** se centra en la comunicación entre procesos en máquinas diferentes, ofreciendo servicios de extremo a extremo.

Un servicio de transporte debe contemplar varias condiciones del entorno de red:

- Potencialmente conecta muchas máquinas diferentes.
- Requiere de establecimiento y finalización de conexión explícitos.
- Potencialmente lidia con diferentes **RTT (Round Trip Time)**.
- Requiere mecanismos adaptivos para **timeouts**.
- Potencialmente enfrenta largos retardos en la red.
- Requiere estar preparado para el arribo de paquetes muy antiguos.
- Potencialmente existe diferente capacidad de recepción en destino.
- Requiere contemplar nodos de diferentes capacidades.
- Potencialmente maneja diferente capacidad de red.
- Requiere estar preparado para lidiar con congestión de red.

1.2 Máquina de Estados de un Servicio de Transporte

En un modelo de manejo de conexiones, tanto el **Cliente** como el **Servidor** se representan mediante máquinas de estados. Cada máquina cambia de estado según eventos como la llegada de paquetes, la solicitud de conexión o la terminación de la misma:



1.2.1 Estados del Cliente

- **Reposo (Idle):** Estado inicial. No hay conexión establecida ni pendiente. Se transita a **Establecimiento activo pendiente** al iniciar la conexión.
- **Establecimiento activo pendiente (Active Open Pending):** Se envía la solicitud de conexión al servidor y se espera la respuesta. Si llega la confirmación, se pasa a **Establecida**. En caso de timeout, puede reintentarse.
- **Establecida (Established):** La conexión se ha establecido y se puede intercambiar TPDUs de manera bidireccional.
- **Desconexión activa pendiente (Active Close Pending):** El cliente solicita cerrar la conexión y espera confirmación del servidor. Al recibirla, vuelve a **Reposo**.

1.2.2 Estados del Servidor

- **Reposo (Idle/Listening):** Espera solicitudes de conexión entrantes. Puede atender múltiples clientes.
- **Establecimiento pasivo pendiente (Passive Open Pending):** Ha recibido una solicitud de conexión y espera confirmación final para pasar a **Establecida**.
- **Establecida (Established):** Conexión abierta y lista para intercambio de datos. Mantiene información de la conexión.
- **Desconexión pasiva pendiente (Passive Close Pending):** Ha recibido la solicitud de cierre del cliente y envía confirmación. Tras la confirmación final del cliente, vuelve a **Reposo**.

1.3 Protocolos End-to-End en subredes de datagramas

Los protocolos de transporte en subredes de datagramas se apoyan en la **capa de Red**, que funciona con un servicio de **mejor esfuerzo (best-effort)**. Esto implica que:

- Pueden descartarse mensajes.
- Los mensajes pueden llegar desordenados.
- Puede entregarse más de una copia de un mismo mensaje.
- Los mensajes están limitados a un tamaño finito.
- La entrega puede ocurrir después de un tiempo arbitrariamente largo.

Los servicios comunes **End-to-End** que se desean ofrecer incluyen:

- Garantía de entrega de mensajes (**confiabilidad**).
- Entrega de mensajes en el mismo orden en que fueron enviados.

- Entrega de a lo sumo una copia de cada mensaje.
- Soporte para mensajes arbitrariamente largos.
- Soporte de **sincronización**.
- Permitir al receptor controlar el **flujo de datos** del transmisor.
- Soportar múltiples procesos de nivel aplicación en un receptor.

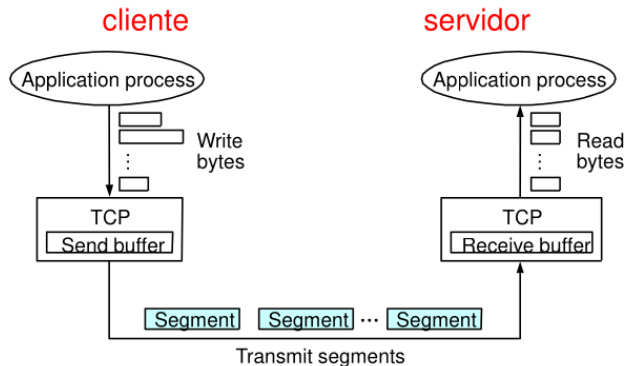
2 TCP (Transmission Control Protocol)

TCP es un protocolo de transporte **orientado a conexión** que garantiza la entrega confiable de datos entre aplicaciones en diferentes máquinas. Se basa en el concepto de **flujo de bytes**, en el cual la aplicación escribe bytes en un buffer de envío, TCP empaqueta esos bytes en segmentos, los envía a la red y la aplicación receptora lee bytes desde un buffer de recepción.

Características principales

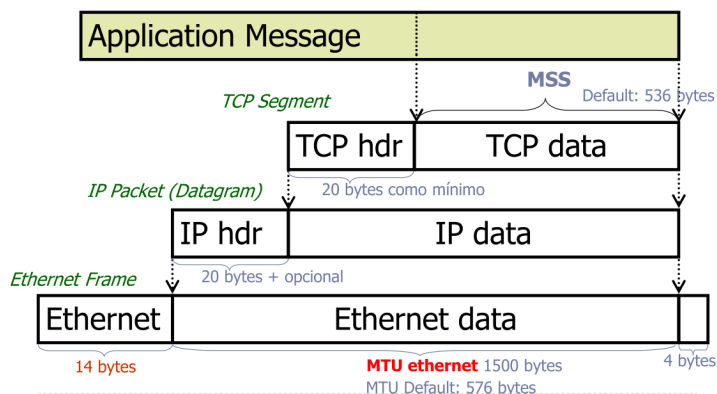
- **Orientado a conexión:** Antes de enviar datos, TCP establece una conexión entre cliente y servidor mediante un **3-way handshake**. La liberación de la conexión se realiza típicamente mediante un **4-way handshake**.
- **Flujo de bytes:** La aplicación interactúa con TCP escribiendo y leyendo bytes; TCP se encarga de empaquetarlos en segmentos y asegurar su entrega.
- **Full Duplex:** Permite dos flujos de bytes independientes entre cliente y servidor.
- **Control de flujo:** Evita que el transmisor inunde al receptor con más datos de los que puede procesar.
- **Control de congestión:** Evita que el transmisor sobrecargue la red.
- **Confiabilidad:** TCP garantiza la entrega de datos mediante:
 - Acknowledgements (ACKs)
 - Checksums
 - Números de secuencia para detectar datos perdidos o desordenados
 - Retransmisión de datos después de un timeout
 - Reordenamiento de segmentos desordenados

2.1 Flujo de datos en TCP



- La aplicación escribe bytes en el **send buffer** del TCP.
- TCP envía esos bytes como **segmentos** a través de la red.
- Los segmentos llegan al **receive buffer** del receptor, donde la aplicación puede leer los bytes.

2.2 Segmentación y MSS



- TCP divide los datos en segmentos con un tamaño máximo llamado **MSS (Maximum Segment Size)**.
- El **MSS por defecto** es 536 bytes (adaptable según la MTU de la red).
- Cada segmento TCP incluye un **TCP header** mínimo de 20 bytes, encapsulado dentro de un **IP packet**, y luego dentro de un **Ethernet frame**.

2.3 Conexión TCP

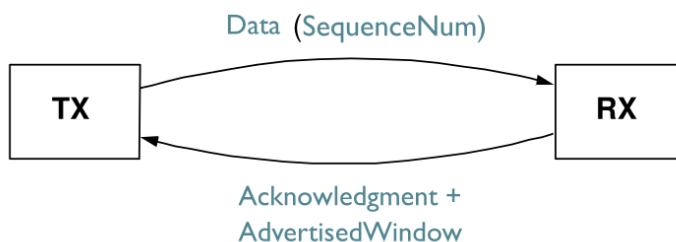
Cada conexión TCP se identifica mediante la **4-tupla**:

SrcPort, SrcIPAddr, DstPort, DstIPAddr

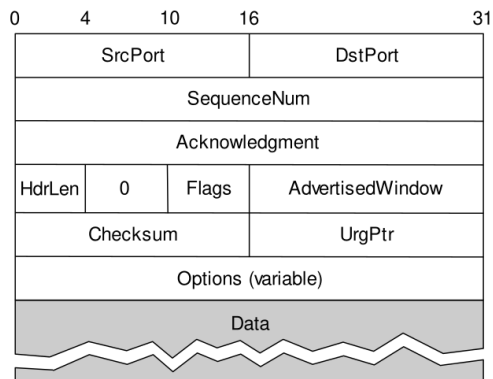
TCP utiliza una **ventana deslizante** para el control de flujo, la cual se basa en:

- **SequenceNum**: Número de secuencia del primer byte en un segmento.
- **Acknowledgment**: Número del siguiente byte que el receptor espera recibir.
- **AdvertisedWindow**: Tamaño de ventana disponible en el receptor.

El flujo de datos se representa así:



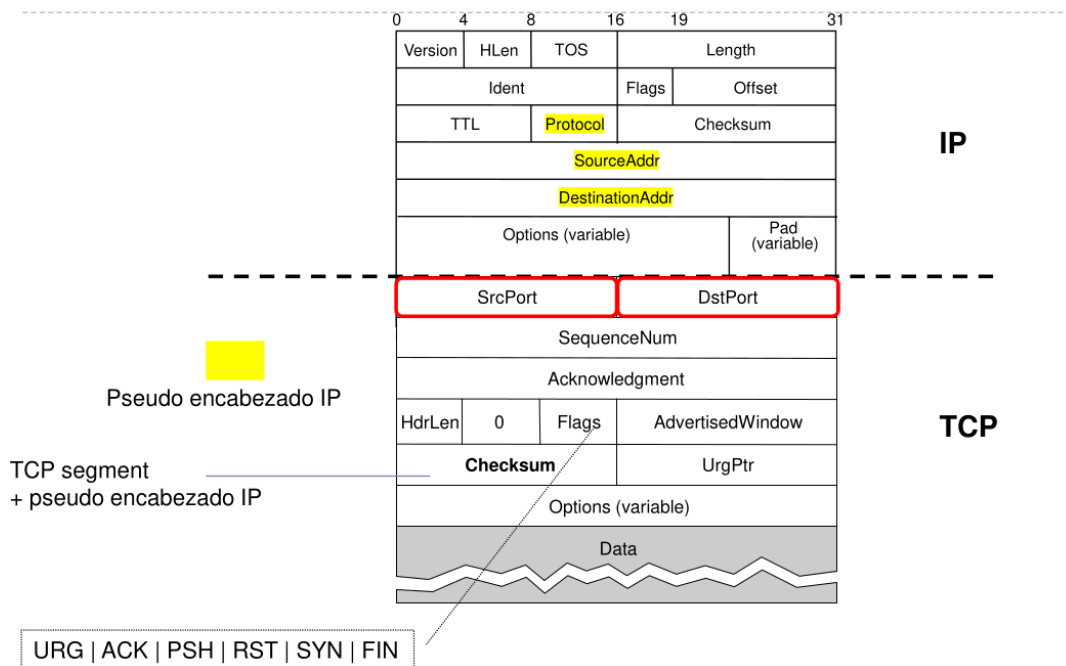
2.4 Formato del Header TCP



El TCP header incluye los siguientes campos:

- **Puerto fuente y destino (16 bits):** Identifican los puntos finales locales de la conexión.
- **Número de secuencia (32 bits):** Identifica el primer byte de datos del segmento.
- **Número de reconocimiento (32 bits):** Indica el siguiente byte que espera el receptor.
- **Longitud de cabecera (HdrLen, 4 bits):** Indica cuántas palabras de 32 bits están en la cabecera.
- **Flags:** Controlan la conexión:
 - **URG:** Indica que el campo Puntero a Urgente está en uso.
 - **ACK:** Indica que el número de reconocimiento es válido.
 - **PSH:** “PUSHed data”, entrega inmediata de datos a la aplicación.
 - **RST:** Resetea una conexión que se volvió inconsistente.
 - **SYN:** Establecimiento de conexión (requiere handshake).
 - **FIN:** Liberación de conexión, indica que no hay más datos.
- **Tamaño de ventana:** Bytes que pueden enviarse desde el último byte reconocido.
- **Checksum:** Verificación de integridad de los datos y encabezado.
- **UrgPtr:** Puntero a datos urgentes si URG=1.
- **Opciones (variable):** MSS, timestamps, etc.
- **Data:** Datos de la aplicación.

2.5 Contexto de encapsulamiento



TCP se apoya en IP, que añade un pseudo-encabezado para el cálculo de checksum y entrega los segmentos como datagramas. Cada segmento TCP se encapsula dentro de:

- IP Packet (IP header + TCP segment)
- Ethernet Frame (si la capa física es Ethernet)

2.6 Emulación del Stream de Bytes

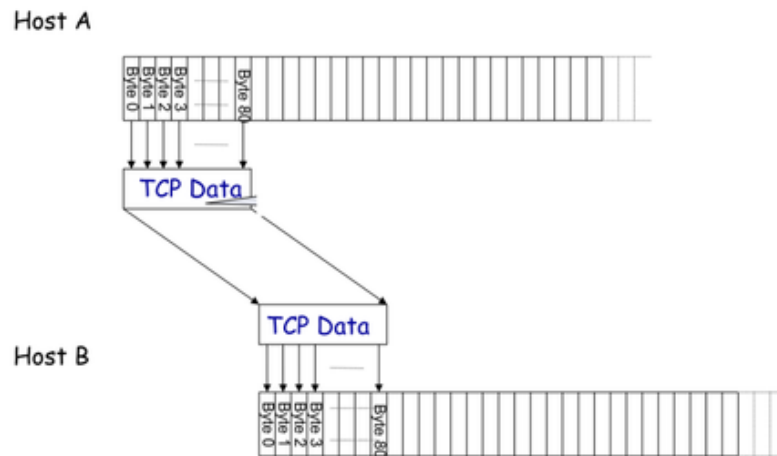
TCP es un protocolo orientado a conexión que permite a las aplicaciones enviar y recibir un **flujo continuo de bytes**, conocido como **stream de bytes**. A diferencia de los protocolos sin conexión como UDP, TCP organiza los datos de la aplicación en **segmentos**, asegurando confiabilidad, orden y control de flujo.

Aunque la aplicación percibe un flujo continuo de bytes, TCP internamente envía estos datos en unidades llamadas **segmentos TCP**. Cada segmento contiene un conjunto de bytes extraídos del flujo, junto con su encabezado que incluye información de secuencia, control de flujo y verificación de errores.

El envío de un segmento TCP desde un host (**Host A**) hacia otro (**Host B**) ocurre bajo las siguientes condiciones:

- **Segmento lleno:** Cuando el buffer de transmisión alcanza el tamaño máximo de segmento (MSS, Maximum Segment Size), se envía automáticamente un segmento completo.
- **Timeout:** Si el buffer no está lleno pero ha transcurrido un tiempo definido sin que se haya enviado un segmento, TCP fuerza el envío para garantizar que los datos no queden retenidos indefinidamente.
- **Push de la aplicación:** En algunas situaciones, la aplicación solicita que los datos sean entregados inmediatamente, mediante el uso del flag **PSH** en el segmento TCP. Esto asegura que la información llegue al receptor sin esperar a que el segmento se llene o expire el timeout.

Flujo conceptual

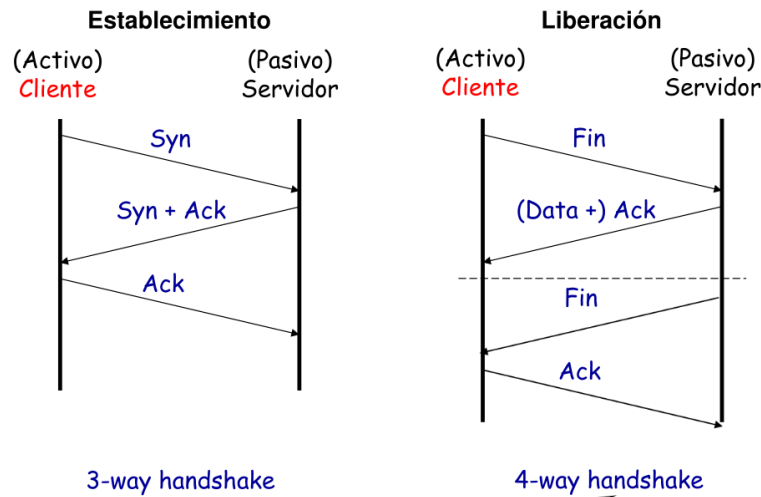


El flujo de bytes se puede conceptualizar de la siguiente manera:

- La aplicación escribe bytes en el **send buffer** de TCP.
- TCP organiza esos bytes en segmentos según las condiciones mencionadas.
- Cada segmento se envía hacia el receptor, donde TCP en el host destino reconstruye el flujo de bytes en el **receive buffer**, garantizando que la aplicación pueda leer los datos en el orden correcto y de manera confiable.

De este modo, TCP logra **simular un flujo continuo de bytes** sobre una red que en realidad transporta paquetes discretos, proporcionando a las aplicaciones un servicio **confiable, ordenado y controlado**.

2.7 Establecimiento y Liberación de Conexión



2.7.1 Establecimiento de Conexión

TCP es un protocolo orientado a conexión, por lo que antes de intercambiar datos se debe **establecer la conexión** entre el cliente y el servidor. El establecimiento se realiza mediante un **3-way handshake**:

1. **SYN (Cliente → Servidor):** El cliente inicia la conexión enviando un segmento con el flag SYN activado.
2. **SYN + ACK (Servidor → Cliente):** El servidor reconoce la solicitud y responde con un segmento que activa los flags SYN y ACK.
3. **ACK (Cliente → Servidor):** Finalmente, el cliente confirma la recepción del SYN del servidor con un ACK, completando la conexión.

Este proceso asegura que ambos extremos conocen el estado de la conexión y los números de secuencia iniciales para el flujo de datos.

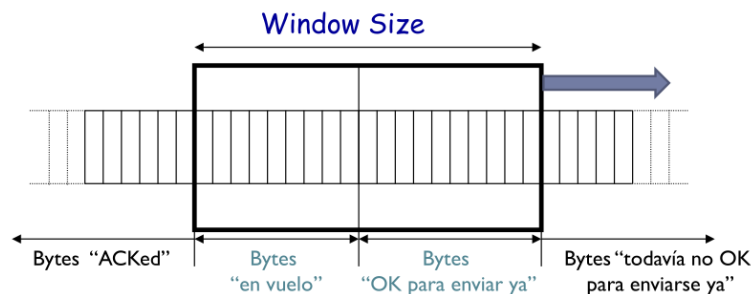
2.7.2 Liberación de Conexión

La **liberación de la conexión** se realiza mediante un **4-way handshake**:

1. **FIN (Activo) – Cliente:** El cliente indica que no enviará más datos.
2. **ACK (Pasivo) – Servidor:** El servidor reconoce la recepción del FIN.
3. **FIN (Pasivo) – Servidor:** El servidor también indica que ha terminado de enviar datos.
4. **ACK (Activo) – Cliente:** El cliente confirma la recepción del FIN del servidor y se completa la liberación.

En el [RFC 793](#) se describen varios escenarios, incluyendo cierres simultáneos y conexiones que finalizan mientras aún hay datos en tránsito, pero el caso ideal se realiza como se describió.

2.8 Ventana Deslizante

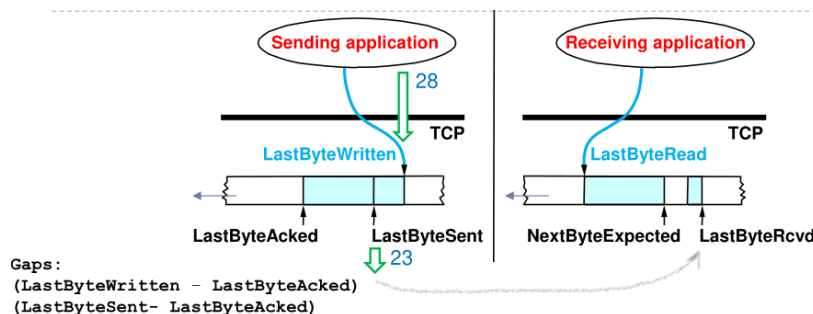


TCP implementa una **ventana deslizante** para garantizar confiabilidad, control de flujo y entrega de datos en orden. La ventana deslizante organiza los bytes de un flujo TCP en varias categorías:

- Bytes “ACKed”: Confirmados por el receptor.
- Bytes “en vuelo”: Enviados pero aún no reconocidos.
- Bytes “OK para enviar ya”: Dentro de la ventana que el receptor ha anunciado.
- Bytes “todavía no OK para enviarse ya”: Fuera de la ventana anunciada, no pueden enviarse.

La política de retransmisión usual es **Go Back N**, donde si un segmento se pierde, todos los segmentos posteriores deben retransmitirse. El tamaño de ventana (**Window Size**) es anunciado por el receptor, típicamente entre 4 KB y 8 KB durante la fase de establecimiento.

2.8.1 Ventana Deslizante del Emisor



En el lado del emisor se manejan tres punteros principales:

- **LastByteAcked**: Último byte confirmado por el receptor.
- **LastByteSent**: Último byte enviado al receptor.
- **LastByteWritten**: Último byte generado por la aplicación.

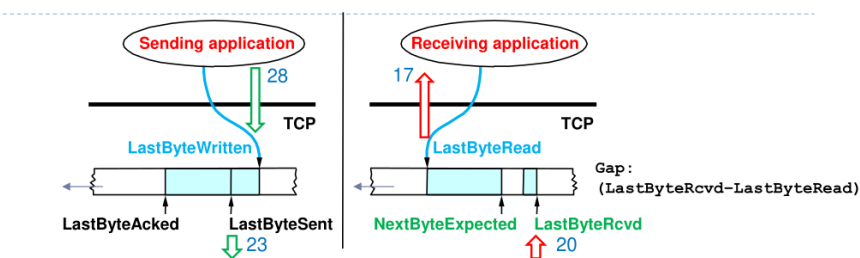
Las relaciones son:

$$LastByteAcked \leq LastByteSent \leq LastByteWritten$$

Los bytes se almacenan en buffer entre LastByteAcked y LastByteWritten:

- A la izquierda de LastByteAcked: ya confirmados.
- A la derecha de LastByteWritten: aún no generados por la aplicación.

2.8.2 Ventana Deslizante del Receptor



El receptor mantiene punteros para controlar la recepción y entrega de bytes:

- **LastByteRcvd**: Último byte recibido (aunque puede estar desordenado).
- **NextByteExpected**: Siguiendo byte que la aplicación espera recibir en orden.
- **LastByteRead**: Último byte leído por la aplicación.

Reglas importantes:

$$LastByteRead < NextByteExpected \leq LastByteRcvd + 1$$

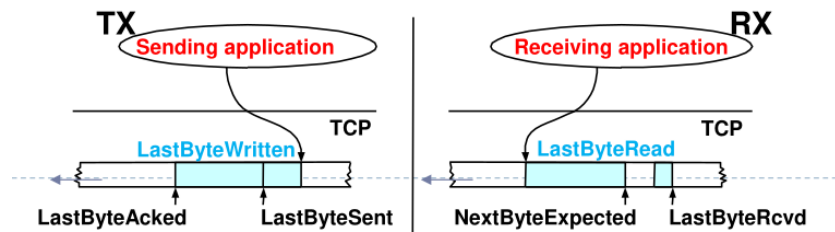
- Los bytes que se reciben fuera de orden se encolan hasta que todos los precedentes lleguen.
- La aplicación solo puede leer los bytes **en orden**, garantizando consistencia del flujo.
- Los bytes pendientes se encuentran entre **NextByteExpected** y **LastByteRcvd**, formando la “brecha” que se espera completar.

2.9 Control de Flujo

TCP implementa un **control de flujo** para garantizar que el emisor no envíe más datos de los que el receptor puede procesar, evitando desbordes de buffer y pérdida de información. Este mecanismo se basa en los tamaños de los buffers y en la ventana anunciada por el receptor.

Buffers

- **Buffer de envío (MaxSendBuffer):** Capacidad máxima de bytes que el emisor puede mantener pendientes de confirmación.
- **Buffer de recepción (MaxRcvBuffer):** Capacidad máxima de bytes que el receptor puede almacenar mientras espera que la aplicación los lea.



Lado Receptor (RX)

El receptor controla la cantidad de datos que puede aceptar según la siguiente regla:

$$(\text{LastByteRcvd} - \text{LastByteRead}) \leq \text{MaxRcvBuffer}$$

Se define la **ventana anunciada** (AdvertisedWindow) como:

$$\text{AdvertisedWindow} = \text{MaxRcvBuffer} - (\text{LastByteRcvd} - \text{LastByteRead})$$

De esta manera, el receptor informa al emisor cuántos bytes adicionales puede recibir sin riesgo de desbordar su buffer.

Lado Emisor (TX)

El emisor respeta la ventana anunciada y las capacidades de su propio buffer:

$$(\text{LastByteSent} - \text{LastByteAcked}) \leq \text{AdvertisedWindow}$$

$$\text{EffectiveWindow} = \text{AdvertisedWindow} - (\text{LastByteSent} - \text{LastByteAcked})$$

$$(\text{LastByteWritten} - \text{LastByteAcked}) \leq \text{MaxSendBuffer}$$

Si la aplicación desea escribir y bytes más de los que permite el buffer:

$$(\text{LastByteWritten} - \text{LastByteAcked}) + y > \text{MaxSendBuffer}$$

el emisor bloquea temporalmente la escritura hasta que haya espacio disponible en el buffer.

2.9.1 Funcionamiento

- El receptor envía **segmentos ACK** en respuesta a la llegada de datos, indicando el progreso de la lectura y la disponibilidad de buffer.
- El emisor persiste en la transmisión, incluso enviando **1 byte** cuando la ventana anunciada es cero, para garantizar que el flujo se reactive tan pronto como el receptor tenga espacio disponible.

Lado emisor (TX):

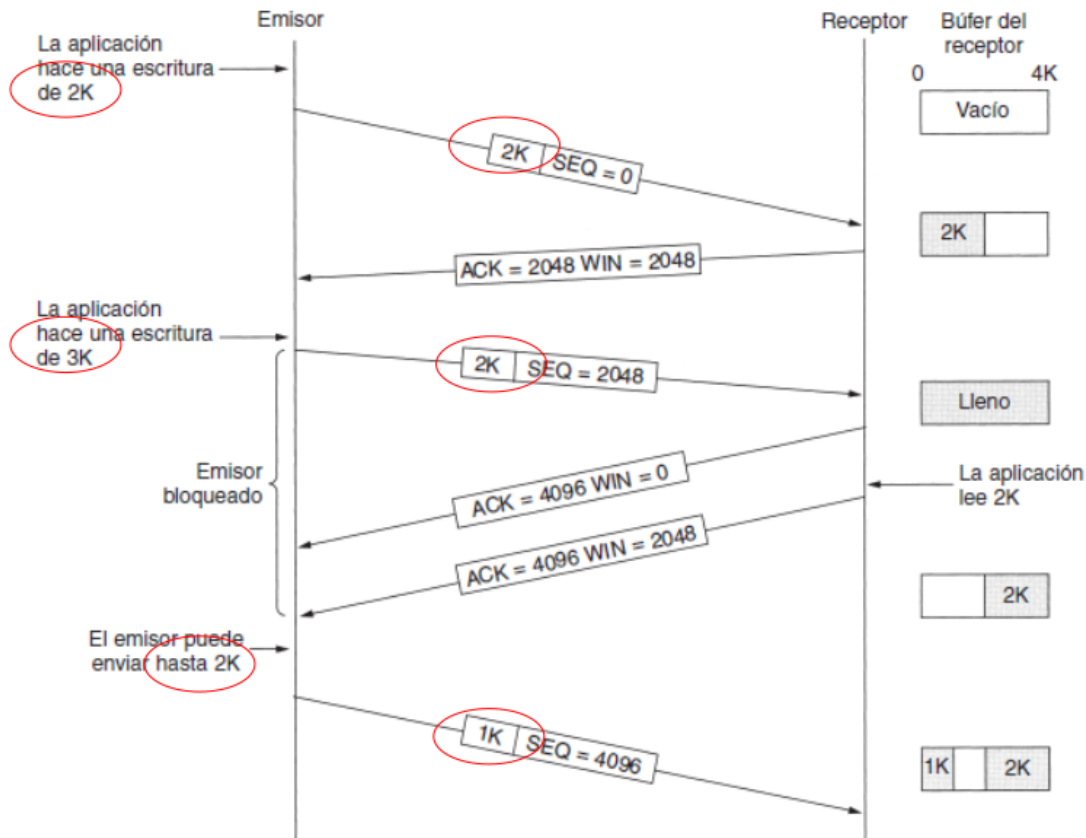
- **LastByteWritten:** Último byte generado por la aplicación.
- **LastByteSent:** Último byte enviado al receptor.
- **LastByteAcked:** Último byte confirmado por el receptor.

Lado receptor (RX):

- **LastByteRcvd:** Último byte recibido (puede estar desordenado).
- **NextByteExpected:** Siguiendo byte que la aplicación espera leer.
- **LastByteRead:** Último byte leído por la aplicación.

Este esquema asegura que **el emisor nunca sobrepase la capacidad del receptor** y que los datos sean entregados de manera ordenada y confiable.

2.10 Advertised Window Management y Números de Secuencia



TCP utiliza un mecanismo de **ventana anunciada (Advertised Window)** para controlar el flujo de datos y garantizar que el emisor no envíe más información de la que el receptor puede procesar. Este mecanismo se integra estrechamente con el **sistema de números de secuencia** que asegura confiabilidad y orden en la entrega de los bytes.

Cada byte de datos enviado en TCP tiene un **número de secuencia (Seq_Num)** que lo identifica de manera única dentro de la conexión. El flujo de datos se maneja de la siguiente manera:

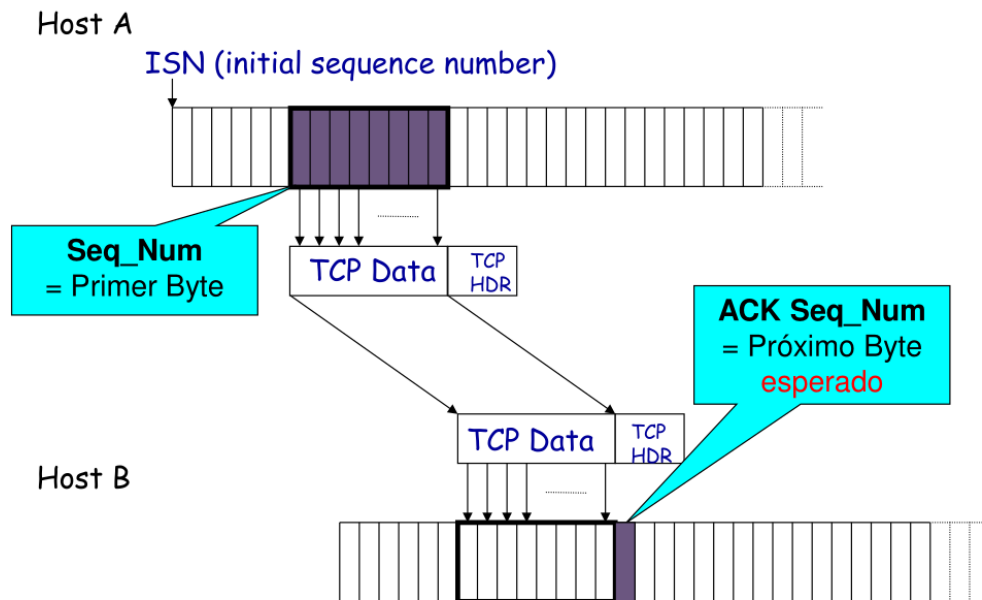
- **ISN (Initial Sequence Number):** Cada host elige un número de secuencia inicial al establecer la conexión. Este número marca el primer byte que se enviará desde ese host.
- Los segmentos TCP incluyen el número de secuencia del primer byte de datos contenido en el segmento.
- El receptor utiliza los números de secuencia para **reconocer (ACK)** los datos recibidos y asegurar que no se pierdan ni se entreguen fuera de orden.

Función de la Ventana Anunciada

La **ventana anunciada (Advertised Window)** indica al emisor cuántos bytes adicionales puede enviar sin sobrepasar la capacidad del buffer del receptor. Conjuntamente con los números de secuencia:

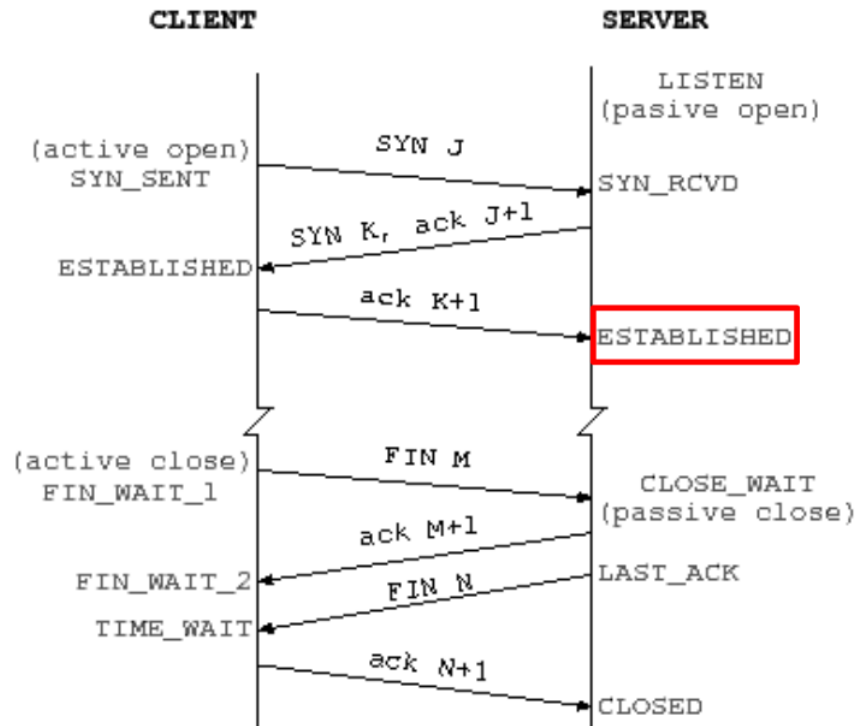
- El receptor informa el **próximo byte esperado (NextByteExpected)** mediante el campo ACK del segmento TCP.
- El emisor puede enviar todos los bytes cuya secuencia esté dentro de la ventana anunciada.
- Los segmentos TCP contienen tanto el **número de secuencia** como la **ventana anunciada**, lo que permite un control de flujo dinámico y seguro.

Ejemplo



1. **Host A** inicia la transmisión con un ISN determinado.
2. Los bytes enviados son numerados secuencialmente: Seq_Num = Primer Byte.
3. **Host B** recibe los datos y responde con un ACK que indica el **próximo byte esperado** (ACK Seq_Num = Próximo Byte esperado).
4. El proceso se repite, moviendo la ventana de recepción y ajustando la cantidad de bytes que el emisor puede enviar, asegurando que los datos lleguen en orden y sin desbordar el buffer del receptor.

2.11 Estados del Modelo TCP



TCP utiliza una **máquina de estados** para administrar la conexión desde su establecimiento hasta su liberación. Cada estado refleja el progreso de la conexión y las acciones que pueden ocurrir ante la llegada de segmentos o llamadas de sistema.

2.11.1 Administración de Conexiones

En un escenario típico, un **cliente se conecta activamente** a un servidor que escucha pasivamente. En las representaciones gráficas de la máquina de estados:

- Las **líneas gruesas** indican el flujo de eventos común: líneas continuas para el cliente y líneas punteadas para el servidor.
- Las **líneas delgadas** representan secuencias de eventos menos comunes o excepcionales.
- Cada línea se asocia con un **evento y una acción**.

Eventos

Los eventos pueden ser:

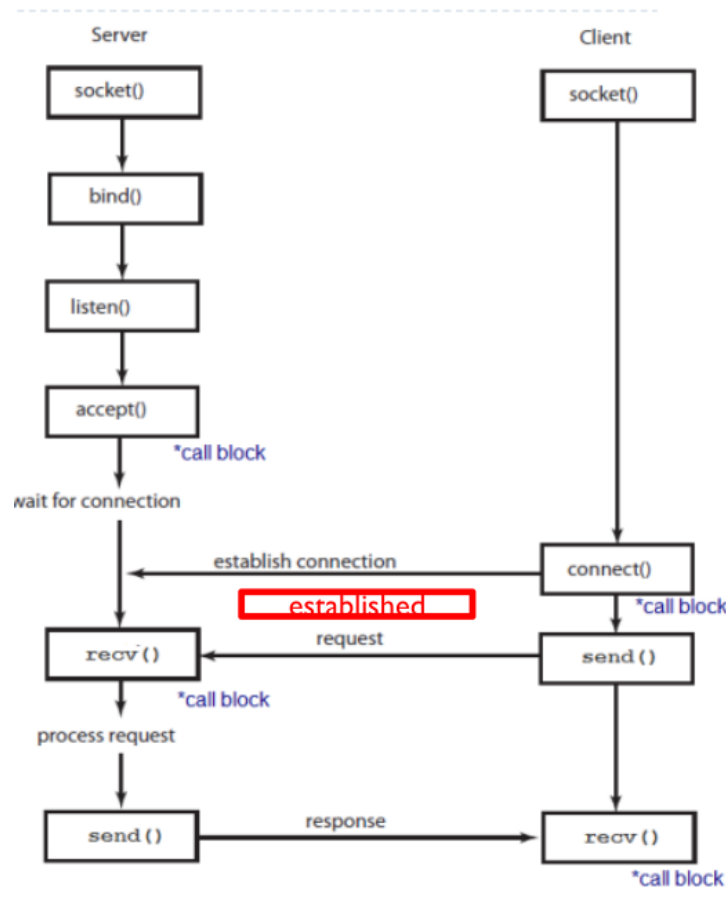
- **Llamadas de sistema** iniciadas por la aplicación: CONNECT, LISTEN, SEND, CLOSE.
- **Llegada de segmentos TCP:** SYN, FIN, ACK, RST.
- **Expiración de temporizadores**, por ejemplo, al superar el doble del tiempo de vida máximo de un paquete.

Acciones

Las acciones asociadas a los eventos pueden ser:

- **Envío de segmentos de control:** SYN, FIN, RST.
- **Ninguna acción**, indicada con un guion (-).
- Comentarios adicionales pueden aparecer entre paréntesis para describir situaciones especiales.

2.12 Primitivas – Sockets Berkeley TCP



Las primitivas típicas utilizadas en TCP con sockets Berkeley incluyen:

- **connect()**: para iniciar una conexión activa desde el cliente.
- **listen()**: para preparar un socket servidor pasivo que acepte conexiones entrantes.
- **send()**: para transmitir datos sobre la conexión establecida.
- **close()**: para iniciar la liberación de la conexión.

Cuando la conexión se encuentra **establecida**, la aplicación puede escribir y leer datos de manera confiable.

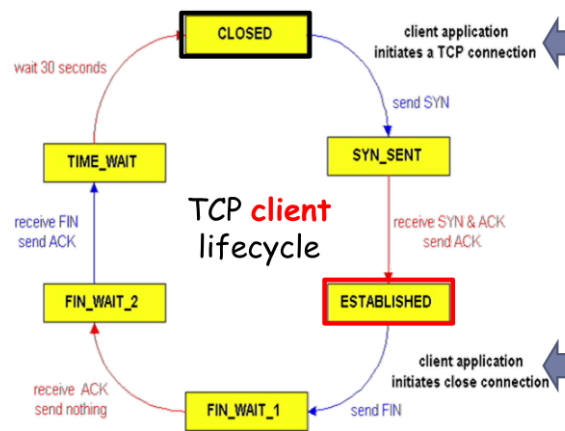
2.12.1 Principales Estados TCP

- **CLOSED**: Estado inicial o final, donde no existe ninguna conexión activa.
- **LISTEN**: Estado del servidor esperando solicitudes de conexión entrantes (SYN). Se utiliza solo en sockets pasivos.
- **SYN_SENT**: Estado del cliente que ha enviado un SYN para iniciar una conexión activa y espera un SYN-ACK del servidor.
- **SYN_RECEIVED**: Estado del servidor que ha recibido un SYN y enviado un SYN-ACK al cliente, esperando la confirmación final (ACK).
- **ESTABLISHED**: La conexión está completamente establecida; ambas partes pueden enviar y recibir datos.
- **FIN_WAIT_1**: Estado del emisor que ha iniciado el cierre enviando un FIN y espera un ACK del receptor.
- **FIN_WAIT_2**: Estado posterior al ACK recibido por el FIN del emisor; espera el FIN del receptor.
- **CLOSE_WAIT**: Estado del receptor que ha recibido un FIN y debe notificar a la aplicación que cierre la conexión.
- **LAST_ACK**: Estado del receptor que ha enviado su FIN y espera el ACK final del emisor para completar la liberación de la conexión.
- **TIME_WAIT**: Estado del emisor tras recibir el FIN-ACK del receptor; espera un tiempo suficiente para asegurarse de que todos los segmentos retrasados se han recibido y no haya duplicados en futuras conexiones.

2.13 Ciclo de Vida en la Máquina de Estados TCP

TCP maneja la conexión mediante una máquina de estados tanto en el cliente como en el servidor. Cada extremo sigue un ciclo de vida específico desde el establecimiento hasta la liberación de la conexión.

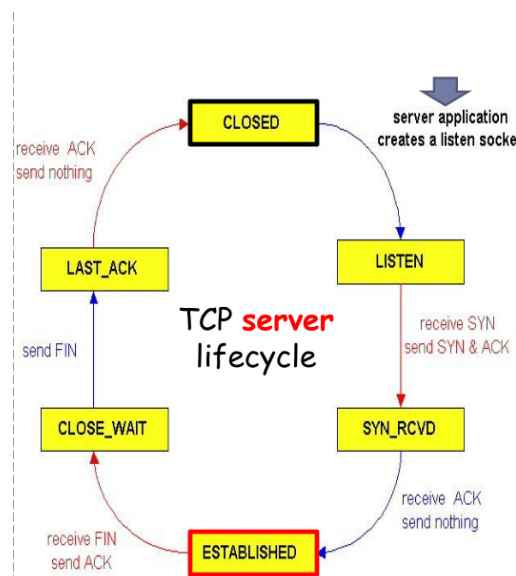
2.13.1 TCP Client Lifecycle



Un **cliente TCP** inicia una conexión de manera activa y atraviesa varios estados hasta que la conexión se cierra completamente:

- **CLOSED**: Estado inicial del cliente. No existe ninguna conexión activa ni pendiente.
- **SYN_SENT**: El cliente envía un segmento **SYN** al servidor para solicitar la conexión. En este estado espera recibir un **SYN-ACK** del servidor.
- **ESTABLISHED**: El cliente recibe el **SYN-ACK** del servidor y envía su **ACK** de confirmación. La conexión queda establecida y se pueden enviar y recibir datos libremente.
- **FIN_WAIT_1**: El cliente ha decidido cerrar la conexión y envía un segmento **FIN**. Espera el **ACK** de este **FIN** por parte del servidor.
- **FIN_WAIT_2**: El cliente ha recibido el **ACK** del **FIN** enviado y ahora espera que el servidor envíe su propio **FIN**.
- **TIME_WAIT**: Tras recibir el **FIN** del servidor y enviar el **ACK** final, el cliente entra en **TIME_WAIT** para asegurarse de que todos los segmentos retrasados en la red se hayan recibido. Después de este tiempo, pasa a **CLOSED**.

2.13.2 TCP Server Lifecycle



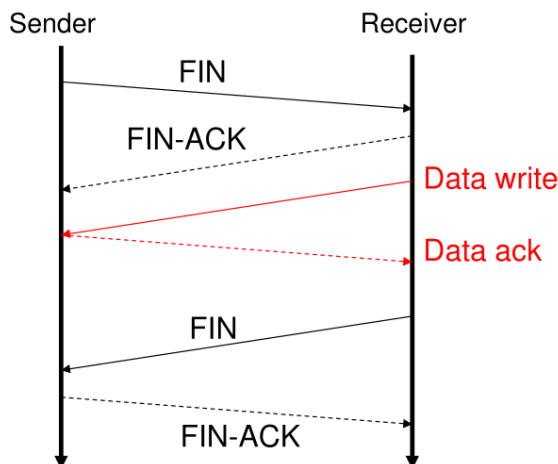
Un **servidor TCP** espera conexiones entrantes y maneja la conexión de manera pasiva:

- **CLOSED:** Estado inicial del servidor. No hay conexiones activas.
- **LISTEN:** El servidor está esperando solicitudes de conexión (SYN) de los clientes. Este es el estado típico de un socket pasivo.
- **SYN_RECEIVED:** El servidor recibe un SYN de un cliente y responde con un SYN-ACK. Espera el ACK final del cliente para completar el establecimiento de la conexión.
- **ESTABLISHED:** Tras recibir el ACK del cliente, la conexión queda establecida. El servidor puede enviar y recibir datos.
- **CLOSE_WAIT:** El servidor recibe un FIN del cliente indicando que éste ha terminado de enviar datos. Debe notificar a la aplicación para cerrar la conexión.
- **LAST_ACK:** El servidor envía su propio FIN para liberar la conexión y espera el ACK final del cliente. Una vez recibido, pasa a CLOSED.

2.14 Liberación de la Conexión TCP (“Tear-down”)

El **proceso de cierre de una conexión TCP** garantiza que todos los datos pendientes se entreguen y que ambos extremos reconozcan que la conexión ha terminado de manera ordenada. Tanto el emisor (*Sender*) como el receptor (*Receiver*) participan activamente en este proceso.

Pasos del Tear-down



1. **FIN del emisor (Sender):** Cuando la aplicación decide terminar la conexión, el emisor envía un segmento FIN al receptor. Este segmento indica que el emisor **ya no tiene más datos para enviar**, pero aún puede recibir datos que estén en tránsito desde el receptor.
2. **ACK del receptor (Receiver):** El receptor recibe el FIN y responde con un ACK, confirmando que recibió la solicitud de cierre. Durante este tiempo, la aplicación en el receptor aún puede leer los datos pendientes que hayan llegado.
3. **FIN del receptor:** Una vez que el receptor ha procesado todos los datos pendientes y decide cerrar la conexión, envía su propio FIN al emisor. Esto indica que ahora el receptor **tampoco tiene más datos para enviar**.
4. **ACK final del emisor:** El emisor recibe el FIN del receptor y responde con un ACK final. En este punto, ambos extremos han confirmado que la conexión ha sido cerrada correctamente.

Notas

- Durante el tear-down, todavía se pueden transmitir y reconocer **datos pendientes**. Por eso los estados como CLOSE_WAIT y FIN_WAIT aseguran que nada se pierda.
- El emisor entra en TIME_WAIT después de enviar el último ACK para asegurarse de que cualquier segmento retrasado en la red se reciba y no se generen duplicados en conexiones futuras.
- Este proceso completo normalmente se conoce como **4-way handshake**, ya que involucra cuatro segmentos de control (FIN, ACK, FIN, ACK) para cerrar la conexión de manera confiable.

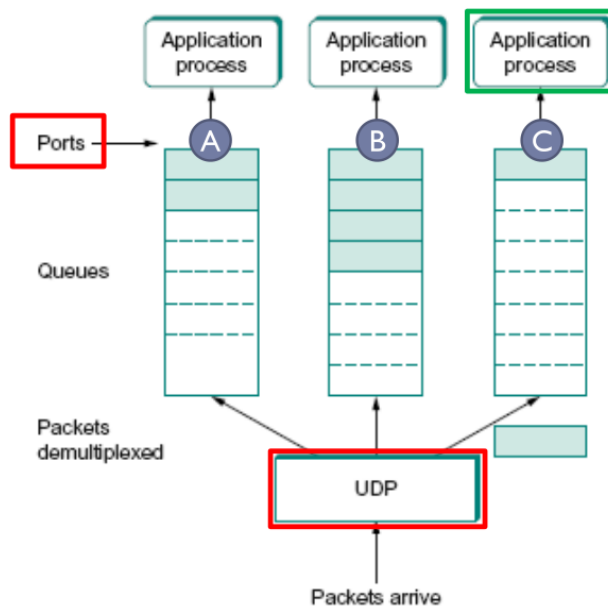
3 UDP (User Datagram Protocol)

UDP (User Datagram Protocol) es un protocolo de la **capa de transporte** del modelo OSI que proporciona un servicio **sin conexión** y de bajo overhead para aplicaciones que requieren rapidez y eficiencia por encima de confiabilidad estricta.

A diferencia de TCP, UDP no establece ni mantiene una conexión entre el emisor y el receptor:

- No hay handshake de establecimiento ni liberación de conexión.
- Cada datagrama UDP es enviado de manera independiente.
- Los datagramas pueden llegar desordenados, duplicados o incluso perderse, ya que UDP **no realiza retransmisiones** ni control de flujo.
- Esta simplicidad permite **baja latencia y alto rendimiento**, ideal para aplicaciones en tiempo real como voz (VoIP), video streaming, o juegos en línea.

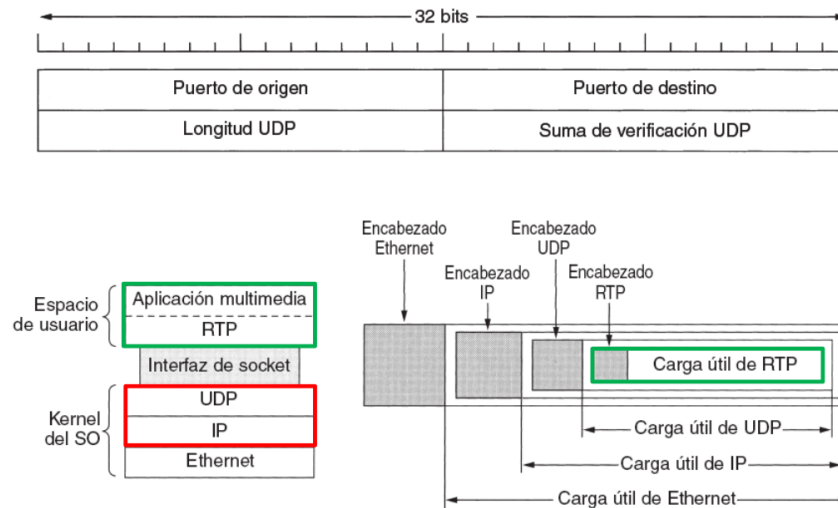
3.1 Multiplexación mediante Puertos



UDP utiliza **puertos** para permitir que múltiples aplicaciones compartan la misma dirección IP:

- Cada datagrama UDP incluye un **Puerto de Origen** y un **Puerto de Destino**, ambos de 16 bits.
- Esto permite que la capa de transporte **multiplexe y demultiplexe** datos entre diferentes aplicaciones y servicios en un mismo host.
- Los puertos bien conocidos permiten identificar servicios estándar, por ejemplo: DNS (53), DHCP (67/68) o RTP (5004/5005).

3.2 Ejemplo: Transporte UDP y Aplicación RTP



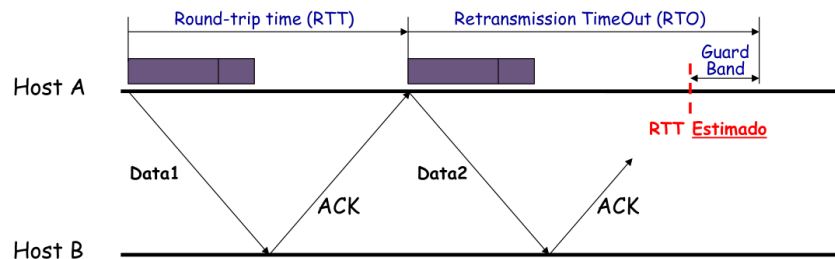
Un caso típico de uso de UDP es en aplicaciones de **RTP (Real-time Transport Protocol)**:

- RTP proporciona transporte de **audio y video en tiempo real** sobre UDP.
- La combinación de UDP y RTP permite minimizar la latencia, ya que no se espera la retransmisión de paquetes perdidos.
- Los datagramas UDP transportan los paquetes RTP, que incluyen información de secuencia y marca temporal para que el receptor pueda reconstruir correctamente el flujo de medios, incluso si algunos paquetes se pierden o llegan desordenados.
- Esta arquitectura es fundamental en aplicaciones como videoconferencias, streaming de música y servicios VoIP.

4 Retransmisión Adaptativa

TCP es un protocolo confiable que asegura la entrega de datos a través de redes potencialmente inestables. Una de sus características más importantes es la **retransmisión adaptativa**, que permite gestionar los tiempos de espera de manera dinámica para evitar retransmisiones innecesarias y optimizar el rendimiento de la red.

Esquema

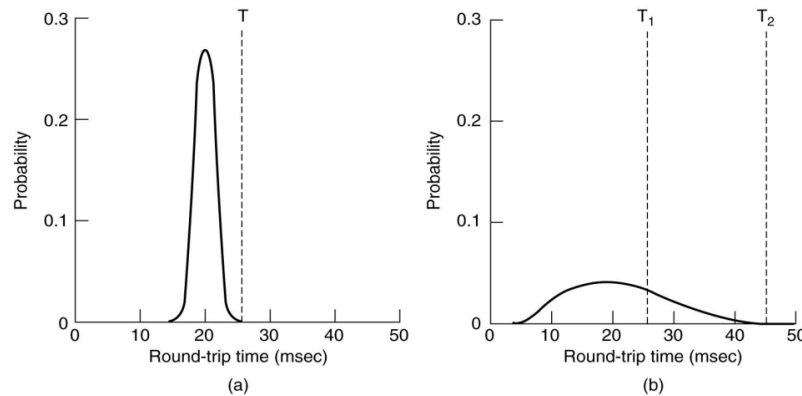


- **Round-Trip Time (RTT)**: Tiempo que tarda un segmento en ir desde el emisor al receptor y recibir su correspondiente ACK.
- **Retransmission Timeout (RTO)**: Tiempo de espera después del cual un segmento no confirmado se retransmite.
- **Guard Band**: Margen adicional de tiempo para cubrir variaciones en el RTT.

Debido a la **variabilidad del RTT**, que puede cambiar por congestión de red o cambios de ruta, TCP necesita un **algoritmo adaptativo** para ajustar los tiempos de retransmisión.

4.1 TCP Timer Management

TCP necesita determinar un **timeout** para retransmisiones. Este timeout depende del **tiempo de ida y vuelta (RTT)** de los segmentos, que es variable. Para modelar esta variabilidad se usan **funciones de densidad de probabilidad (PDF)** que muestran cómo se distribuyen los tiempos de llegada de los ACKs.



a) Función densidad de probabilidad para tiempos de llegada de ACK en Nivel 2

- En el **nivel de enlace de datos**, los ACKs de los segmentos tienen una **distribución relativamente estrecha**.
- Esto significa que, salvo congestión severa o errores, los tiempos de llegada de los ACKs son bastante predecibles.
- La PDF mostraría un **pico pronunciado alrededor del RTT promedio**, indicando que la mayoría de los ACKs llegan cerca del tiempo esperado.

b) Función densidad de probabilidad para tiempos de llegada de ACK en TCP

- En TCP, la situación se complica porque hay **retrasos de red, congestión, cambios de ruta, colas en routers**, etc.
- Como resultado, la PDF de los tiempos de llegada de ACKs en TCP es **más ancha y dispersa**: los ACKs pueden llegar mucho antes o mucho después del RTT promedio.
- Esto implica que un timer fijo sería ineficiente: si se pone demasiado corto, hay retransmisiones innecesarias; si se pone demasiado largo, la recuperación de pérdidas es lenta.

4.2 Algoritmo Original de Retransmisión Adaptativa

El algoritmo inicial de TCP (Jacobson, 1988) funciona de la siguiente manera:

1. Se mide un **SampleRTT** para cada par segmento/ACK recibido.
2. Se calcula un promedio ponderado para estimar el RTT:

$$\text{EstimatedRTT} = \alpha \cdot \text{EstimatedRTT} + \beta \cdot \text{SampleRTT}_i$$

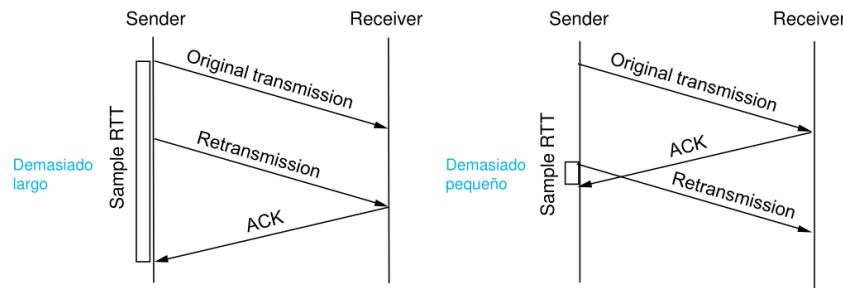
donde $\alpha + \beta = 1$, típicamente $0.8 \leq \alpha \leq 0.9$ y $0.1 \leq \beta \leq 0.2$.

3. El **timeout de retransmisión** se fija en función de este EstimatedRTT:

$$\text{TimeOut} = 2 \times \text{EstimatedRTT}$$

Este enfoque permite que el timeout se ajuste según la dinámica de la red.

4.3 Problemas del Algoritmo Básico y Solución de Karn/Partridge (KP87)



El algoritmo original presentaba problemas de incertidumbre cuando los ACKs correspondían a paquetes retransmitidos:

- Si un paquete se retransmite, no se puede usar su RTT para ajustar el EstimatedRTT.
- KP87 propone **ignorar los SampleRTT de paquetes retransmitidos**.
- Además, se aplica un **exponential backoff**, duplicando el timeout tras cada retransmisión, evitando que la red se sature.

4.4 Algoritmo de Jacobson/Karels (JK88)

Jacobson y Karels mejoraron la estimación de timeout incorporando la **varianza del RTT**, logrando un control más preciso:

1. Calcular la diferencia entre el SampleRTT y el EstimatedRTT:

$$\text{Diff} = \text{SampleRTT} - \text{EstRTT}$$

2. Actualizar la estimación y la desviación:

$$\text{EstRTT} = \text{EstRTT} + \delta \cdot \text{Diff}$$

$$\text{Dev} = \text{Dev} + \delta \cdot (|\text{Diff}| - \text{Dev})$$

donde δ es un factor de suavizado ($\delta \approx 1/8$).

3. El **timeout de retransmisión** se calcula considerando la desviación:

$$\text{TimeOut} = \mu \cdot \text{EstRTT} + \phi \cdot \text{Dev}$$

con $\mu = 1$ y $\phi = 4$. Este valor se adapta dinámicamente según la variabilidad del RTT.

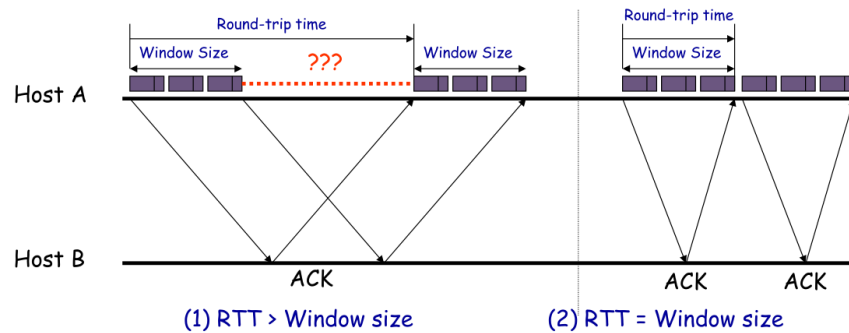
Observaciones

- La precisión de estos algoritmos depende de la **granularidad del reloj** del sistema (por ejemplo, 500 ms en sistemas Unix-like).
- Un mecanismo de timeout preciso no solo evita retransmisiones innecesarias, sino que también **ayuda a controlar la congestión** de la red.
- Estas técnicas fueron fundamentales para el desarrollo de los algoritmos modernos de control de congestión en TCP.

5 Uso del Ancho de Banda, Sockets y Ventana Deslizante

TCP es un protocolo confiable de transporte que no solo asegura la entrega de datos, sino que también **gestiona eficientemente el uso del ancho de banda** de la red. La combinación de la **ventana deslizante** y la estimación del **round-trip time (RTT)** permite mantener la red “llena” sin saturarla.

5.1 Ventana Deslizante (Sliding Window)



La **ventana deslizante** es el mecanismo mediante el cual TCP controla cuántos bytes pueden estar “en vuelo” entre el emisor y el receptor sin haber sido reconocidos:

- Cada extremo anuncia un **AdvertisedWindow**, que indica cuántos bytes adicionales puede recibir sin desbordar su buffer.
- Esta ventana tiene un tamaño de 16 bits en TCP, lo que permite anunciar hasta 64 KB de datos pendientes de recepción.
- El producto **Delay** $\times$ **Bandwidth** indica la cantidad óptima de datos que debería estar en tránsito para mantener el enlace ocupado sin interrupciones.

RTT y Ventana

El **round-trip time (RTT)** afecta directamente la eficiencia del enlace:

1. Si $RTT > Window\ Size$, el emisor debe esperar ACKs antes de enviar más datos, lo que puede provocar subutilización del ancho de banda.
2. Si $RTT = Window\ Size$, el enlace se mantiene lleno y se optimiza la transferencia de datos.

5.2 Ejemplo: Delay $\times$ Bandwidth y Ventana TCP

La siguiente tabla muestra cómo **TCP y otros protocolos de transporte deben dimensionar la ventana de transmisión** para mantener el enlace de red ocupado eficientemente.

Conceptos clave

- **Ancho de banda (Bandwidth):** Capacidad máxima del enlace para transferir datos, medida en bits por segundo (bps).
- **Delay $\times$ Bandwidth (Producto Retardo $\times$ Ancho de Banda):** Cantidad óptima de datos que debería estar “en vuelo” entre el emisor y el receptor para aprovechar completamente el enlace. Se calcula como:

$$\text{Datos en vuelo} = \text{Ancho de banda} \times \text{RTT}$$

Por ejemplo, si el enlace es de 10 Mbps y el RTT es de 100 ms:

$$10\text{ Mbps} \times 0.1\text{ s} = 1\text{ Mb} = 125\text{ KB}$$

Esto indica la cantidad de datos que TCP debería enviar antes de esperar ACKs, manteniendo el “caño” de la red lleno.

5.3 Interpretación de la tabla

- Cada fila indica cuántos bytes **deberían estar en tránsito** para mantener el enlace lleno sin interrupciones.
- Por ejemplo, para un enlace **Ethernet** de 10 Mbps y RTT de 100 ms, la ventana óptima es 64 KB. Esto coincide con el tamaño máximo de ventana TCP clásica (16 bits = 64 KB), permitiendo saturar el enlace sin esperar ACKs continuamente.
- Para enlaces más rápidos, como STS-24 (1.2 Gbps), el producto es mucho mayor (14.8 MB), mostrando que **ventanas TCP estándar no son suficientes** sin extensiones como *TCP window scaling*.

Enlace	Ancho de banda	Delay $\times$ Bandwidth
T1	1.5 Mbps	18 KB
Ethernet	10 Mbps	64 KB
T3	45 Mbps	122 KB
FDDI	100 Mbps	549 KB
STS-3	155 Mbps	1.8 MB
STS-12	622 Mbps	7.4 MB
STS-24	1.2 Gbps	14.8 MB

Este ejemplo permite entender la relación entre velocidad de la red, RTT y tamaño de ventana TCP:

- Si la ventana es demasiado pequeña, la red no se utiliza completamente (**subutilización**).
- Si la ventana es adecuada según Delay $\times$ Bandwidth, se mantiene el “caño lleno” y se optimiza el **throughput**.

5.4 Uso de Sockets

En TCP, los **sockets** son la interfaz que permiten a las aplicaciones leer y escribir bytes de manera confiable sobre la red:

- La aplicación escribe bytes en el **send buffer** del socket.
- TCP segmenta estos bytes y los envía al receptor respetando la ventana deslizante y el control de flujo.
- Los bytes recibidos se almacenan en el **receive buffer**, y la aplicación puede leerlos en orden mediante llamadas al socket.

Este modelo garantiza un **flujo continuo de datos**, aprovechando el ancho de banda disponible y evitando congestión o pérdida de datos.